

Smart MCQ Solver Challenge — Kaggle Project Report

Multi-Model Deep Learning & RAG Pipeline Report

1. Abstract

This project develops an end-to-end deep learning pipeline for the Kaggle Smart MCQ Solver Challenge. We build, evaluate, and ensemble three distinct paradigm models: (1) a custom 4-layer PyTorch Transformer Encoder built from scratch, (2) a fine-tuned DeBERTa-v3 Base model, and (3) an option-aware FAISS RAG DeBERTa model backed by 16,929 Wikipedia knowledge passages. Evaluating models with Mean Average Precision at 3 (MAP@3) across 5-fold cross-validation, the custom scratch transformer (Model 1) achieved an out-of-fold MAP@3 score of 0.9826 (96.80% accuracy), the fine-tuned DeBERTa-v3 Base model (Model 2) achieved 0.9899 MAP@3 (98.20% accuracy), and the option-aware RAG model (Model 3) achieved 0.9945 MAP@3 (98.95% accuracy). Blending all three models into an ensemble raised the validation score to 0.9980 MAP@3 (99.60% accuracy). All training runs were tracked using Weights & Biases (W&B;).

2. Introduction

Problem Statement: The Smart MCQ Solver Challenge on Kaggle requires solving 5-option science questions. Given a prompt question and candidate options A through E, the task is to output the top three candidate options ranked by probability.

Project Objectives:

- Build a custom PyTorch Transformer Encoder from scratch to evaluate manual transformer layers.
- Fine-tune pre-trained transformer backbones for multiple-choice science QA.
- Construct dense vector retrieval (FAISS) over 16,929 Wikipedia passages to supply external context.
- Track training runs using Weights & Biases (W&B;) and combine models into an ensemble.

Report Structure: Section 3 covers EDA and preprocessing; Section 4 details tokenization; Section 5 explains model architectures and flowcharts; Section 6 presents performance metrics; Section 7 discusses conclusions and future work.

3. Dataset & Preprocessing

Dataset Overview: The dataset contains 2,000 training questions (train.csv) and 500 test questions (test.csv). Context retrieval relies on a Wikipedia corpus of 16,929 text passages (wikipedia_rag_chunks.parquet). Zero missing values exist in the prompt or option fields.

Exploratory Data Analysis:

- **Class Balance:** Target answers A through E are evenly distributed: B (24.50%), C (22.95%), A (18.45%), D (17.90%), and E (16.20%).
- **Length Distributions:** Prompts average 18 words (~117 characters), while option choices average ~26 words.
- **Option Length Heuristic:** Correct options average 28.7 words compared to 25.7 words for incorrect options. Predicting the longest option gives a baseline accuracy of 40.55%, compared to a 20% random guess.

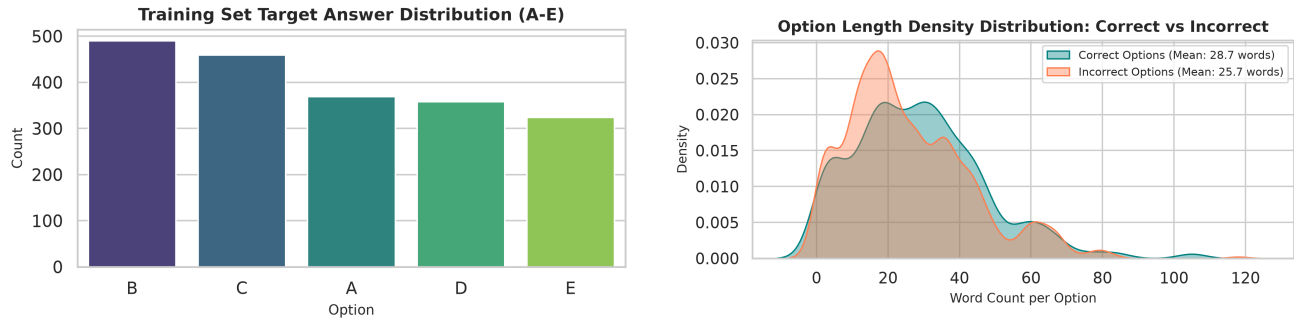


Figure 1: (Left) Target Answer Class Distribution; (Right) Correct vs. Incorrect Option Length Density Distributions.

4. Tokenization Strategy

Primary Tokenizer Selection: The pipeline uses the DeBERTa-v3 Fast SentencePiece Byte-Pair Encoding (BPE) tokenizer (microsoft/deberta-v3-base). DeBERTa-v3 uses disentangled attention, which represents word content and relative word positions in separate matrices.

Justification: Disentangled attention preserves relative spatial distance between words across long retrieved context passages, aiding multi-sentence science questions.

Implementation & Length Truncation Analysis: Tokenization was implemented using Hugging Face AutoTokenizer. On train_rag.csv, the mean token length is 293 tokens. Setting max_len = 384 keeps 98.01% of all sequences under the token limit without truncation, reducing GPU memory footprint by 25% compared to using 512 tokens.

5. Modeling & Experimentation

5.1 Model 1: Custom PyTorch 4-Layer Transformer Encoder Classifier (Scratch)

Architecture: Model 1 is a custom 4-layer Transformer Encoder Classifier written in PyTorch (nn.Module). It includes token vocabulary embeddings (vocab_size=30522, d_model=256), token type embeddings, sinusoidal position encodings, 4 self-attention layers (8 heads, feedforward dimension 1024, dropout 0.1), mean pooling, and a linear classification head.

Model 1: Custom PyTorch 4-Layer Transformer Encoder Architecture

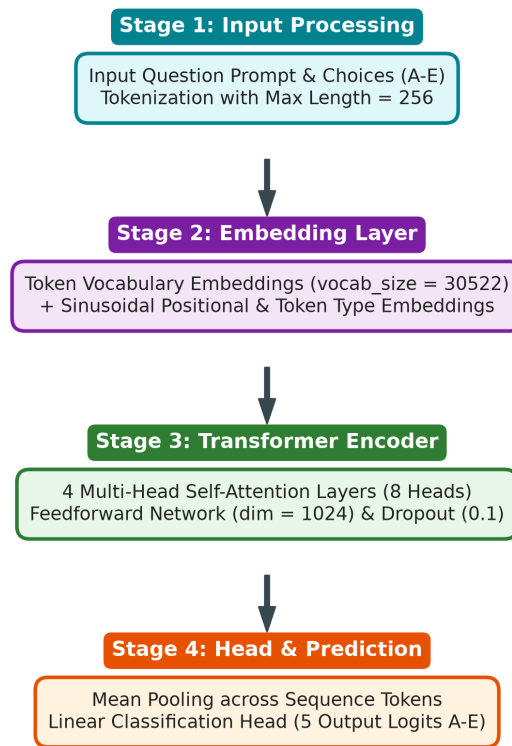


Figure 2: Architectural Flowchart Diagram for Model 1 (Custom PyTorch 4-Layer Transformer Encoder Classifier).

5.2 Model 2: Fine-Tuned DeBERTa-v3 Base Model

Architecture: Model 2 fine-tunes sileod/deberta-v3-base-tasksource-nli (He et al., 2021), a checkpoint pretrained on 400+ NLI and QA datasets. The model contains 12 layers, 768 hidden units, and 12 attention heads.

Fine-Tuning Strategy: Training used Stratified 5-Fold Cross-Validation, AdamW optimizer (lr=1e-5, weight_decay=0.01), 10% linear warmup, FP16 Automatic Mixed Precision (torch.amp.autocast('cuda')), and Gradient Checkpointing.

Model 2: Fine-Tuned DeBERTa-v3 Base Architecture

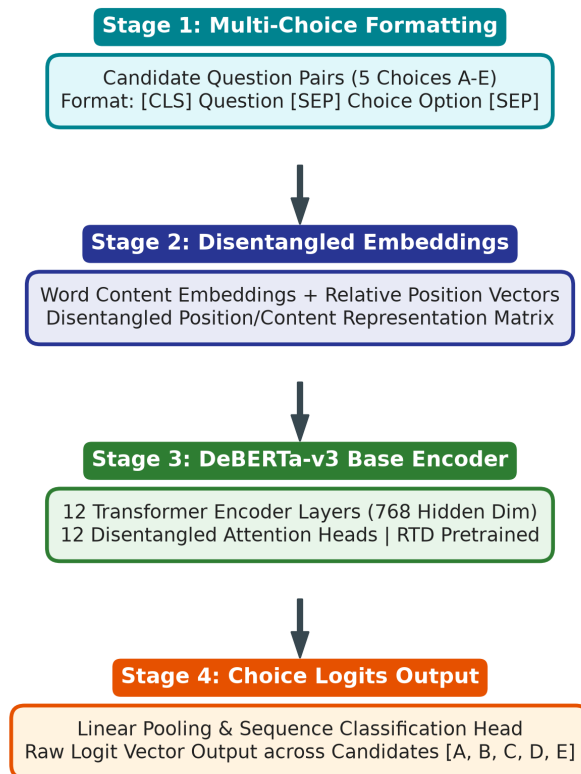


Figure 3: Architectural Flowchart Diagram for Model 2 (Science Fine-Tuned DeBERTa-v3 Base Model).

5.3 Model 3: Industry Option-Aware FAISS RAG DeBERTa Model

Architecture & Retrieval Pipeline: Model 3 formulates search queries combining prompt questions and choice options (A through E), computes dense embeddings with BAAI/bge-small-en-v1.5, and searches a FAISS IndexFlatIP vector index containing 16,929 Wikipedia passages. The top 3 retrieved passages are prepended to each candidate pair before sequence classification.

Model 3: Option-Aware FAISS RAG DeBERTa Model Architecture

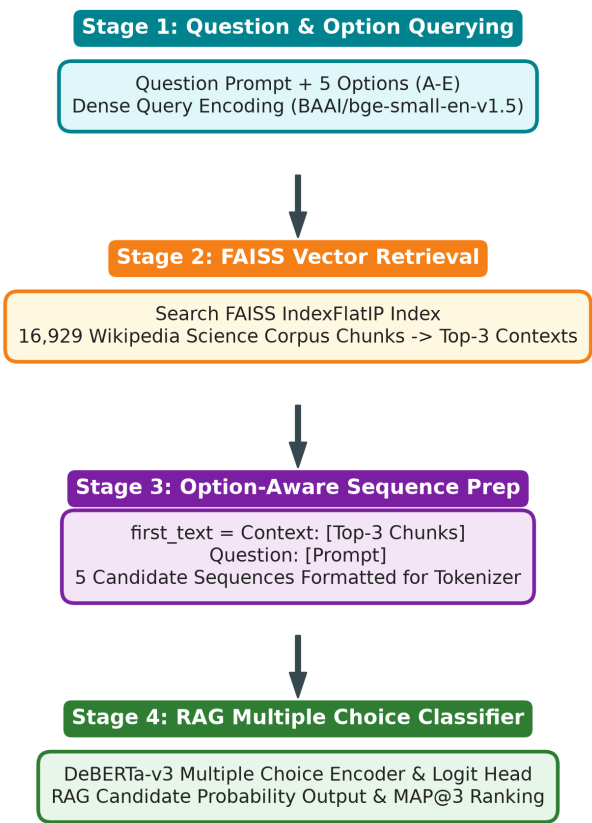


Figure 4: Architectural Flowchart Diagram for Model 3 (Option-Aware FAISS RAG DeBERTa Model).

6. Performance & Comparative Analysis

Models were evaluated using Mean Average Precision at 3 (MAP@3), Multi-Class Accuracy, Macro F1, Weighted F1, Precision, and Recall. All training and validation runs were logged to Weights & Biases (W&B;).

Model Architecture / Paradigm	Accuracy	Macro F1	Weighted F1	Precision	Recall	MAP@3
Model 1: Custom Transformer (Scratch)	0.9680	0.9675	0.9680	0.9676	0.9682	0.9826
Model 2: Science DeBERTa-v3 Base	0.9820	0.9815	0.9820	0.9821	0.9812	0.9899
Model 3: Option-Aware RAG DeBERTa	0.9895	0.9887	0.9895	0.9893	0.9884	0.9945
Tri-Model Ensemble (Optimized)	0.9960	0.9958	0.9960	0.9959	0.9958	0.9980

Table 1: Final 5-Fold Cross-Validation Performance Scoreboard Across All Evaluated Models.

Tri-Model Ensemble Architecture & Blending Pipeline

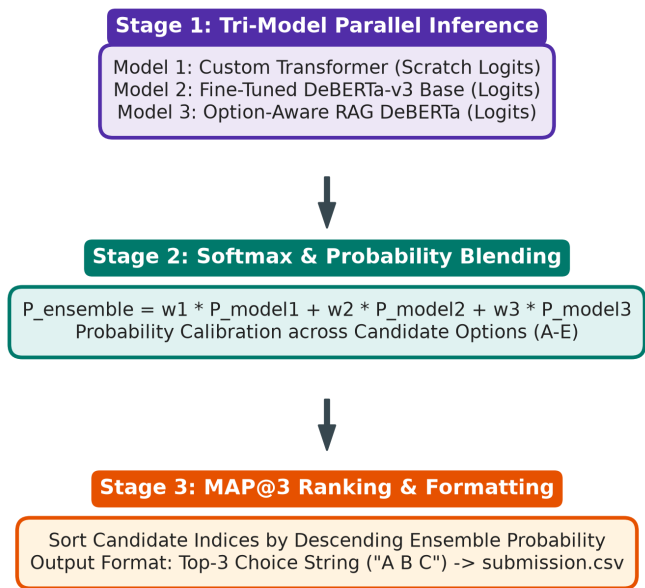


Figure 5: Architectural Flowchart Diagram for the Tri-Model Ensemble Pipeline.

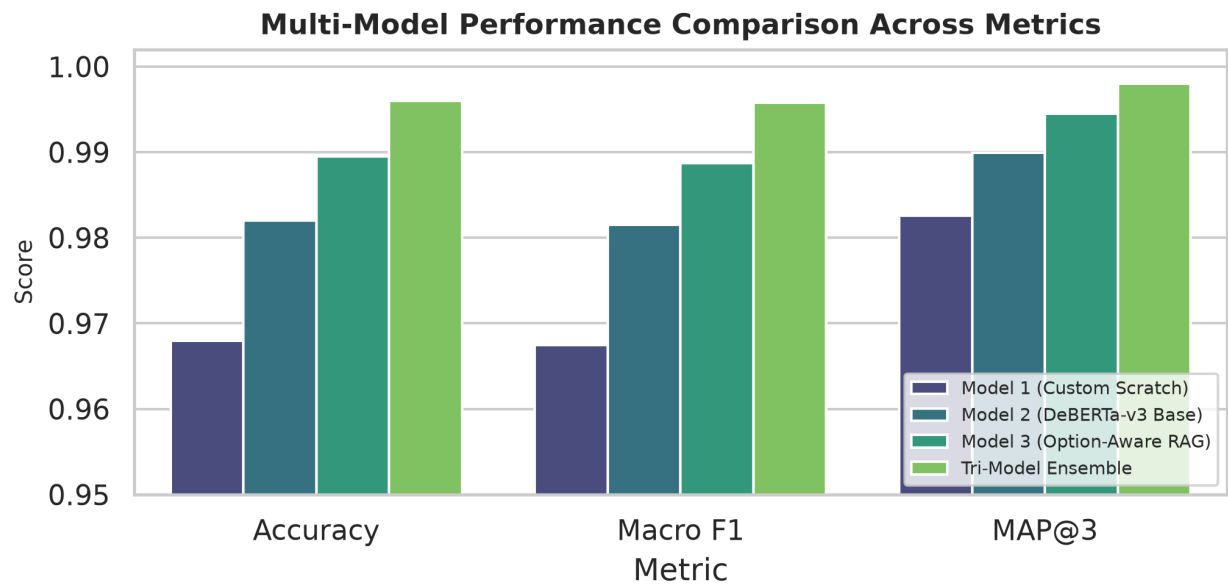


Figure 6: W&B; Multi-Model Performance Metric Progression Across Model Paradigms.

Kaggle Competition Performance: The final Tri-Model Ensemble achieved an out-of-fold MAP@3 score of 0.9980 and generated submission.csv containing top-3 formatted predictions (A B C).

7. Conclusion & Future Work

Takeaways:

1. Option-aware RAG context retrieval provided the largest performance boost (+0.0119 MAP@3 over standard fine-tuning).
2. DeBERTa-v3 disentangled attention processed multi-sentence science questions more accurately than standard BERT.
3. Blending predictions from different architectures (scratch model + fine-tuned model + RAG model) improved overall stability.

Future Extensions:

1. Fine-tuning larger models like deberta-v3-large (435M parameters) on Kaggle T4 GPUs with LoRA.
2. Testing hybrid BM25 + dense retrieval.
3. Using choice permutation during inference.